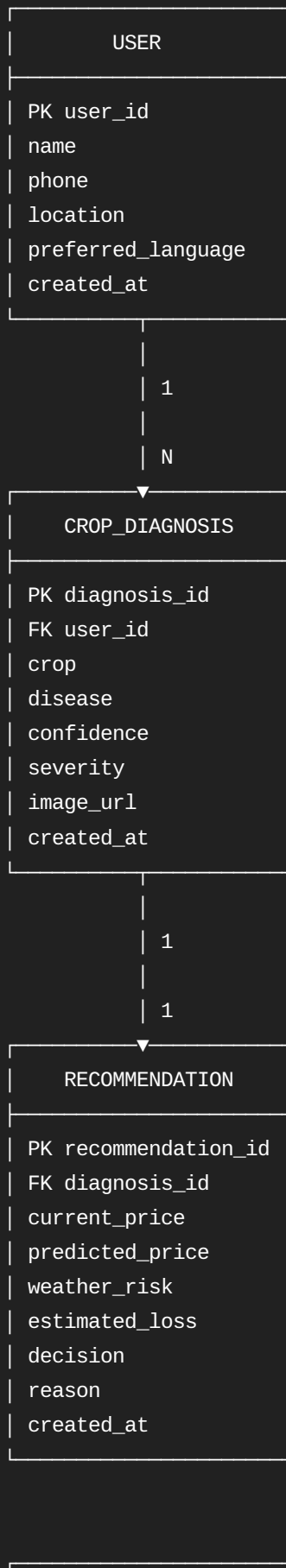


Yes. For your current scope, I'd keep the ER diagram **clean and focused** rather than adding tables for every API/model.

ER Diagram — AI Farm Companion



MARKET_PRICE
PK price_id
crop
market
location
price
date

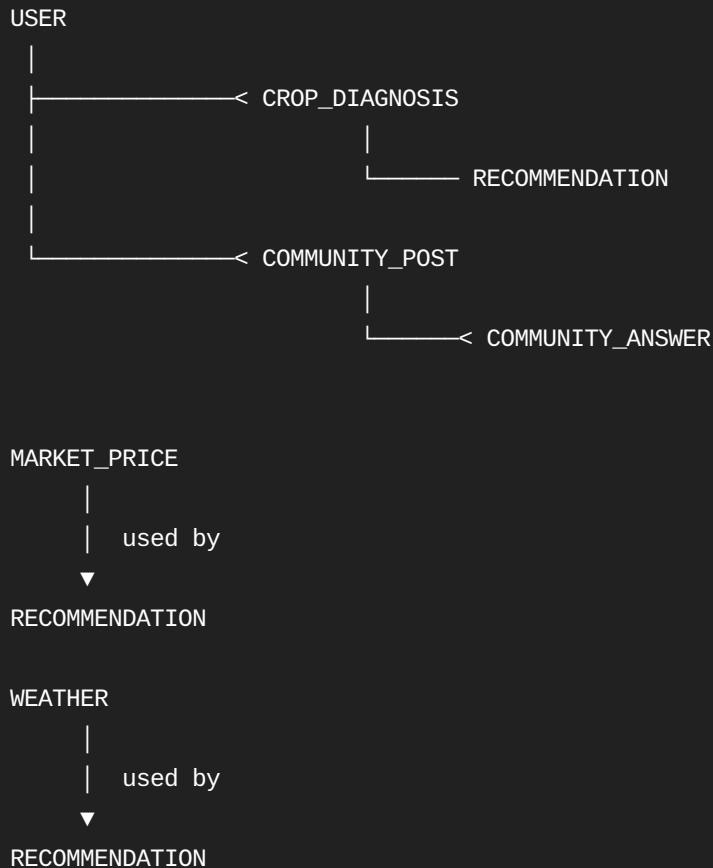
WEATHER
PK weather_id
location
date
temperature
humidity
rainfall
rain_probability

COMMUNITY_POST
PK post_id
FK user_id
crop
question
created_at

1
N

COMMUNITY_ANSWER
PK answer_id
FK post_id
FK user_id
answer
is_ai_generated
created_at

Relationships



One small architectural correction

I wouldn't make `WEATHER` and `MARKET_PRICE` directly dependent on a particular user.

They're external/environmental data:

- Market price → crop + market + date
- Weather → location + date

The **recommendation engine** combines those with the user's crop diagnosis.

So conceptually:

User → Diagnosis → Recommendation

while:

Market + Weather → Recommendation

This keeps your database normalized and makes it easier to reuse the same market/weather data for many farmers.

For your hackathon, I'd start with just **7 tables**:

1. `users`
2. `crop_diagnoses`
3. `recommendations`
4. `market_prices`
5. `weather`
6. `community_posts`
7. `community_answers`

You **don't need separate tables for the disease model, LLM, translation system, or price prediction model**. Those are application/ML components, not database entities.

If you're putting this in your hackathon PPT, this **7-table ERD is the version I'd present**.